

Experiment 'p_4_3_a_extract_uppercase' Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_4_3_a_extract_uppercase

Natural language statement: null

Method signature: p_4_3_a_extract_uppercase(s: string) returns (uppercase: string)

Ensures

- forall c :: c in uppercase ==> 'A' <= c <= 'Z'
- forall c :: c in s && 'A' <= c <= 'Z' ==> c in uppercase

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_4_3_a_extract_uppercase(s: string) returns (uppercase: string)`

The method should respect the following contract:

`ensures forall c :: c in uppercase ==> 'A' <= c <= 'Z', ensures forall c :: c in s && 'A' <= c <= 'Z' ==> c in uppercase`

Produce and show only the Dafny body of this method, including the curly braces that surround it. Do not show the signature nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnLmWyQr2PQMAKQzrnfDpAN7zIWDR

Model: gpt-5.5-2026-04-23

Created at: 1780654172

Finish reason: stop

Usage: 312 tokens in, and 8491 tokens out

```
{
  uppercase := "";
  var i := 0;
  while i < |s|
    invariant 0 <= i <= |s|
    invariant forall c :: c in uppercase ==> 'A' <= c <= 'Z'
    invariant forall j :: 0 <= j < i && 'A' <= s[j] <= 'Z' ==> s[j] in uppercase
  {
    if 'A' <= s[i] <= 'Z' {
      var previous := uppercase;
      assert forall c :: c in previous ==> 'A' <= c <= 'Z';
      assert forall j :: 0 <= j < i && 'A' <= s[j] <= 'Z' ==> s[j] in previous;
      uppercase := previous + [s[i]];

      forall c | c in uppercase
        ensures 'A' <= c <= 'Z'
      {
        assert uppercase == previous + [s[i]];
        assert exists k :: 0 <= k < |uppercase| && uppercase[k] == c;
        var k :| 0 <= k < |uppercase| && uppercase[k] == c;
```

```

    if k < |previous| {
        assert uppercase[k] == previous[k];
        assert previous[k] == c;
        assert c in previous;
        assert 'A' <= c <= 'Z';
    } else {
        assert |uppercase| == |previous| + 1;
        assert k == |previous|;
        assert k - |previous| == 0;
        assert uppercase[k] == [s[i]][k - |previous|];
        assert [s[i]][0] == s[i];
        assert c == s[i];
        assert 'A' <= c <= 'Z';
    }
}

forall j | 0 <= j < i + 1 && 'A' <= s[j] <= 'Z'
ensures s[j] in uppercase
{
    assert uppercase == previous + [s[i]];
    if j < i {
        assert s[j] in previous;
        assert s[j] in uppercase;
    } else {
        assert j == i;
        assert s[j] == s[i];
        assert s[j] in [s[i]];
        assert s[j] in uppercase;
    }
}

} else {
forall j | 0 <= j < i + 1 && 'A' <= s[j] <= 'Z'
ensures s[j] in uppercase
{
    if j < i {
        assert s[j] in uppercase;
    } else {
        assert j == i;
        assert s[j] == s[i];
        assert 'A' <= s[i] <= 'Z';
        assert false;
    }
}

}
i := i + 1;
}

forall c | c in s && 'A' <= c <= 'Z'
ensures c in uppercase
{
    assert exists j :: 0 <= j < |s| && s[j] == c;
}

```

```

    var j :| 0 <= j < |s| && s[j] == c;
    assert i == |s|;
    assert 0 <= j < i;
    assert s[j] == c;
    assert 'A' <= s[j] <= 'Z';
    assert s[j] in uppercase;
    assert c in uppercase;
  }
}

```

Final Program

```

method p_4_3_a_extract_uppercase(s: string) returns (uppercase: string)
  ensures forall c :: c in uppercase ==> 'A' <= c <= 'Z'
  ensures forall c :: c in s && 'A' <= c <= 'Z' ==> c in uppercase
{
  uppercase := "";
  var i := 0;
  while i < |s|
    invariant 0 <= i <= |s|
    invariant forall c :: c in uppercase ==> 'A' <= c <= 'Z'
    invariant forall j :: 0 <= j < i && 'A' <= s[j] <= 'Z' ==> s[j] in uppercase
  {
    if 'A' <= s[i] <= 'Z' {
      var previous := uppercase;
      assert forall c :: c in previous ==> 'A' <= c <= 'Z';
      assert forall j :: 0 <= j < i && 'A' <= s[j] <= 'Z' ==> s[j] in previous;
      uppercase := previous + [s[i]];

      forall c | c in uppercase
        ensures 'A' <= c <= 'Z'
      {
        assert uppercase == previous + [s[i]];
        assert exists k :: 0 <= k < |uppercase| && uppercase[k] == c;
        var k :| 0 <= k < |uppercase| && uppercase[k] == c;
        if k < |previous| {
          assert uppercase[k] == previous[k];
          assert previous[k] == c;
          assert c in previous;
          assert 'A' <= c <= 'Z';
        } else {
          assert |uppercase| == |previous| + 1;
          assert k == |previous|;
          assert k - |previous| == 0;
          assert uppercase[k] == [s[i]][k - |previous|];
          assert [s[i]][0] == s[i];
          assert c == s[i];
          assert 'A' <= c <= 'Z';
        }
      }
    }
  }
}

```

```

forall j | 0 <= j < i + 1 && 'A' <= s[j] <= 'Z'
  ensures s[j] in uppercase
{
  assert uppercase == previous + [s[i]];
  if j < i {
    assert s[j] in previous;
    assert s[j] in uppercase;
  } else {
    assert j == i;
    assert s[j] == s[i];
    assert s[j] in [s[i]];
    assert s[j] in uppercase;
  }
}
} else {
forall j | 0 <= j < i + 1 && 'A' <= s[j] <= 'Z'
  ensures s[j] in uppercase
{
  if j < i {
    assert s[j] in uppercase;
  } else {
    assert j == i;
    assert s[j] == s[i];
    assert 'A' <= s[i] <= 'Z';
    assert false;
  }
}
}
i := i + 1;
}

forall c | c in s && 'A' <= c <= 'Z'
  ensures c in uppercase
{
  assert exists j :: 0 <= j < |s| && s[j] == c;
  var j :| 0 <= j < |s| && s[j] == c;
  assert i == |s|;
  assert 0 <= j < i;
  assert s[j] == c;
  assert 'A' <= s[j] <= 'Z';
  assert s[j] in uppercase;
  assert c in uppercase;
}
}

```

Total Token Usage

Input tokens: 312

Output tokens: 8491

Reasoning tokens: 7610
Sum of ‘total tokens’: 8803

Experiment Timings

Overall Experiment started at 1780654172426, ended at 1780654353825, lasting 181399ms (181.40 seconds)
Iteration #1 started at 1780654172426, ended at 1780654353823, lasting 181397ms (181.40 seconds)